HEAP Priority Queue
(FAANG Cheat Sheet)

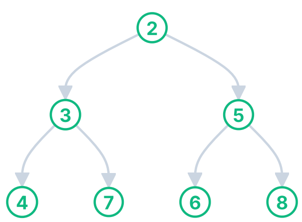
1 WHAT IS A HEAP?

- A Heap is a **Complete Binary Tree**.
- It satisfies the **Heap Property**.
- It is **NOT** a sorted data structure.
- Only the root element is guaranteed to be the minimum (Min Heap) or maximum (Max Heap).
- Heaps are used to implement **Priority Queue**.

2 MIN HEAP VS MAX HEAP

MIN HEAP

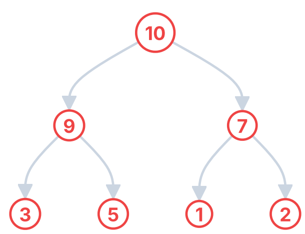
Parent ≤ Children



Smallest element is at the root.

MAX HEAP

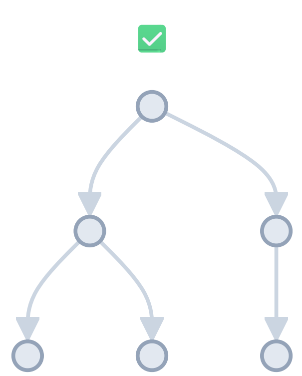
Parent ≥ Children

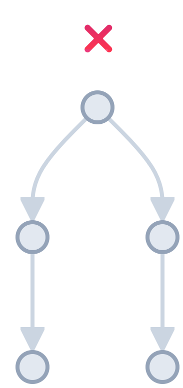


Largest element is at the root.

3 COMPLETE BINARY TREE

All levels are completely filled except possibly the last level, and all nodes in the last level are as far left as possible.





6 JAVA PRIORITYQUEUE

a) Min Heap (Default)

```
PriorityQueue<Integer> minHeap = new
PriorityQueue<>();
// Smallest element at peek()
```

b) Max Heap (3 Ways)

1. Using Collections.reverseOrder() **(Recommended)**

```
PriorityQueue<Integer> maxHeap =
    new PriorityQueue<>
(Collections.reverseOrder());
```

2. Using Comparator (Integer.compare)

```
PriorityQueue<Integer> maxHeap =
    new PriorityQueue<>((a, b) ->
Integer.compare(b, a));
```

3. Using subtraction **(Not recommended - overflow risk)**

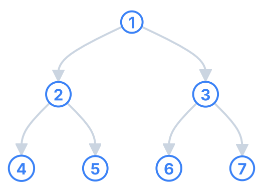
```
PriorityQueue<Integer> maxHeap =
    new PriorityQueue<>((a, b) -> b -
a);
```

c) Common Operations

Method	Description
offer(x)	Inserts element x into heap.
poll()	Removes and returns root element (min or max).
peek()	Returns root element without removing.
size()	Returns number of elements.
isEmpty()	Checks if heap is empty.

4 ARRAY REPRESENTATION (0-INDEXED)

Example Tree



Index Relationships

Parent(i) = (i - 1) / 2
Left(i) = 2 * i + 1
Right(i) = 2 * i + 2

Array Representation

Index	0	1	2	3	4	5	6
Value	**1**	**2**	**3**	**4**	**5**	**6**	**7**

5 TIME COMPLEXITIES

Operation	Min Heap	Max Heap	Why?
offer(x)	O(log n)	O(log n)	Heapify Up
poll()	O(log n)	O(log n)	Heapify Down
peek()	O(1)	O(1)	Root access
size()	O(1)	O(1)	Just length
isEmpty()	O(1)	O(1)	Just length
Build Heap (n elements)	O(n)	O(n)	Bottom-up heapify

⭐ **Build Heap** (Bottom-Up) is O(n) which is better than inserting n elements one by one (O(n log n)).

💡 **Golden Rule: Heaps are all about PRIORITY. Ask: "What**

→ Next Page: Heap

7 IMPORTANT TERMINOLOGY

Root	: Topmost element.
Parent(i)	: (i - 1) / 2
Left Child(i)	: 2 * i + 1
Right Child(i)	: 2 * i + 2
Height	: Longest path from root to leaf.
Leaf Node	: Node with no children.
Heapify	: Process of restoring heap property (Bubble Up or Bubble Down).

8 KEY POINTS TO REMEMBER

- ✓ Heap is a Complete Binary Tree.
- ✓ Heap ≠ Sorted Array.
- ✓ Heaps are used for **Priority**, not order.
- ✓ Choose **Min Heap** or **Max Heap** based on the problem goal.
- ✓ Heap operations are logarithmic time (O(log n)).

9 COMMON MISTAKES

- ❌ Confusing Min Heap with Max Heap.
- ❌ Using wrong comparator.
- ❌ Forgetting that default PriorityQueue in Java is Min Heap.
- ❌ Not handling empty heap before poll/peek.
- ❌ Assuming heap gives sorted order.

element should come out first?" **Patterns Overview**

HEAP - PATTERNS

PAGE 2

Top Interview Patterns with Heaps (Priority Queue)

Max Heap → largest element on top

Min Heap → smallest element on top

PQ → PriorityQueue

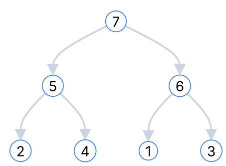
PATTERN 1 REPEATED MAX / MIN RETRIEVAL (e.g., Last Stone Weight)

WHEN TO USE?

- We always take the largest (or smallest) element.
- Do something with it.
- Put the result back.
- Repeat until 1 (or none) left.

HEAP TO USE

Max Heap → **largest**



IDEA / AHA

We always care about the current largest element.

Heap gives it in O(1).

TEMPLATE (JAVA)

```java
PriorityQueue<Integer> pq =
    new PriorityQueue<>(Collections.
    reverseOrder());
// Max Heap

// Build heap
for (int x : arr)
pq.offer(x);

while (pq.size() > 1) {
    int a = pq.poll();
// largest
    int b = pq.poll();
// 2nd largest
    if (a != b)
pq.offer(a - b);
}
return pq.isEmpty() ? 0
: pq.peek();
```

COMPLEXITY

- **Time:** O(n log n)
  (n insertions + up to n polls)
- **Space:** O(n)

EXAMPLES

- LeetCode 1046 Last Stone Weight

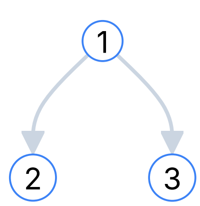
PATTERN 2 TOP-K ELEMENTS (e.g., Kth Largest, Top K Frequent)

WHEN TO USE?

- Need the K largest/smallest elements.
- Or K most frequent, top K, K closest, etc.
- We discard the rest.

HEAP TO USE

Top K Largest → Min Heap (size K)



(Keep K=3 largest seen so

IDEA / AHA

Keep only K best candidates in heap.

If better element comes, replace root (worst among K).

TEMPLATE (JAVA)

→ Top K Largest (Min Heap)

```java
PriorityQueue<Integer>
pq = new
PriorityQueue<>();
for (int x : nums) {
    pq.offer(x);
    if (pq.size() > k)
pq.poll(); // remove
smallest
}
// pq contains top K
largest
```

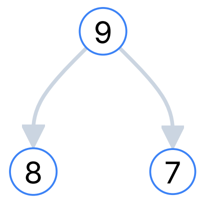
- **Time:** O(n log k)
- **Space:** O(k)

EXAMPLES

- LC 215 Kth Largest Element in Array

far)

Top K Smallest → Max Heap (size K)



(Keep K=3 smallest seen so far)

```
List<Integer> res = new
ArrayList<>(pq);
```

→ Top K Smallest (Max Heap)

```
PriorityQueue<Integer>
pq = new
PriorityQueue<>(

Collections.reverseOrde
r());
for (int x : nums) {
    pq.offer(x);
    if (pq.size() > k)
pq.poll(); // remove
largest
}
// pq contains top K
smallest
```

- LC 347 Top K Frequent Elements
- LC 973 K Closest Points to Origin
- LC 692 Top K Frequent Words

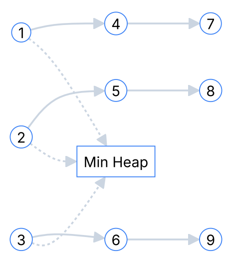
PATTERN 3 MERGE K SORTED LISTS / ARRAYS (e.g., Merge K Sorted Lists)

WHEN TO USE?

- We have K sorted lists/arrays.
- Need to merge into one sorted output.
- Always pick the smallest current candidate.

HEAP TO USE

Min Heap



1 2 3 4 5 6 7 8 9

IDEA / AHA

Put the first element of each list into heap.

Pop smallest node, add its next node (if exists).

TEMPLATE (JAVA)

```
class Node { int val;
Node next; }

PriorityQueue<Node> pq =
new PriorityQueue<>(
    (a, b) -> a.val -
b.val); // Min Heap

// 1. Add head of each
list
for (Node head : lists)
    if (head != null)
pq.offer(head);

Node dummy = new
Node(0), tail = dummy;

while (!pq.isEmpty()) {
    Node cur =
pq.poll();
    tail.next = cur;
tail = tail.next;
    if (cur.next !=
null)
pq.offer(cur.next);
}
return dummy.next;
```

- **Time:** O(N log k)
  (N = total nodes,
  k = number of lists)
- **Space:** O(k)

EXAMPLES

- LeetCode 23 Merge K Sorted Lists
- LeetCode 373 Find K Pairs with Smallest Sums

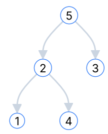
PATTERN 4 TWO HEAPS (MEDIAN FROM DATA STREAM) (e.g., Find Median from Data Stream)

WHEN TO USE?

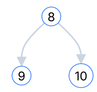
- Need median at any point in a stream of numbers.
- Insertions happen one by one.
- Need O(log n) insert and O(1) median.

HEAP TO USE

Max Heap (Smaller Half)



Min Heap (Larger Half)



IDEA / AHA

💡

Maintain two halves.

1) maxHeap.peek() ≤ minHeap.peek()
2) Size difference ≤ 1

Median is either top of one heap or avg of both.

TEMPLATE (JAVA) - (Add Number)

```java
// maxHeap: left half (max heap)
// minHeap: right half (min heap)
PriorityQueue<Integer> maxHeap =
    new PriorityQueue<>(Collections.reverseOrder());
PriorityQueue<Integer> minHeap = new PriorityQueue<>();

void addNum(int num) {
    if (maxHeap.isEmpty() || num <= maxHeap.peek())
        maxHeap.offer(num);
    else
        minHeap.offer(num);

    // Rebalance
    if (maxHeap.size() > minHeap.size() + 1)
        minHeap.offer(maxHeap.poll());
    else if (minHeap.size() > maxHeap.size())
        maxHeap.offer(minHeap.poll());
}

double findMedian() {
    if (maxHeap.size() == minHeap.size())
        return (maxHeap.peek() + minHeap.peek()) / 2.0;
    return maxHeap.peek();
}
```

- **addNum:** O(log n)
- **findMedian:** O(1)
- **Space:** O(n)

EXAMPLES

- LeetCode 295 Find Median from Data Stream

AHA SUMMARY

✅ What do I care about most right now? Put it on top!

✅ Use heap to discard what I don't need.

✅ Heap ≠ Sorting. Heap gives priority, not full order.

PATTERN RECOGNITION TRIGGERS

- Top K / Kth / Most Frequent / Closest / Smallest / Largest / Merge K / Running Median / Priority / Stream / Scheduler

HEAP - CHEAT SHEET & DECISION GUIDE

Quick Reference for Interviews

PAGE 3

1. HEAP DECISION TREE

Syntax error in text
mermaid version 10.9.6

2. JAVA TEMPLATES (QUICK)

Min Heap (Default)

```
PriorityQueue<Integer> minHeap =
    new PriorityQueue<>();
// smallest element at peek()
```

Max Heap (Using reverseOrder)

```
PriorityQueue<Integer> maxHeap =
    new PriorityQueue<>
(Collections.reverseOrder());
// largest element at peek()
```

Max Heap (Using Comparator)

```
PriorityQueue<Integer> maxHeap =
    new PriorityQueue<>((a, b) ->
Integer.compare(b, a));
// largest element at peek()
```

Top-K Largest (Min Heap of size K)

```
PriorityQueue<Integer> pq = new
PriorityQueue<>();
for (int x : nums) {
    pq.offer(x);
    if (pq.size() > k) pq.poll(); //
remove smallest
}
// pq contains k largest
```

Top-K Smallest (Max Heap of size K)

```
PriorityQueue<Integer> pq =
    new PriorityQueue<>((a, b) ->
Integer.compare(b, a));
for (int x : nums) {
    pq.offer(x);
    if (pq.size() > k) pq.poll(); //
remove largest
}
// pq contains k smallest
```

3. COMPLEXITY TABLE

Operation	Min Heap	Max Heap	Notes
Insert (offer)	O(log n)	O(log n)	Heapify Up
Remove (poll)	O(log n)	O(log n)	Heapify Down
Peek	O(1)	O(1)	Root access
Size	O(1)	O(1)	Just length
isEmpty	O(1)	O(1)	Just length
Build Heap (from n elements)	O(n)	O(n)	Bottom-up heapify

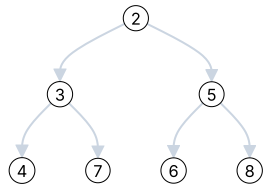
4. KEY INVARIANTS

Min Heap

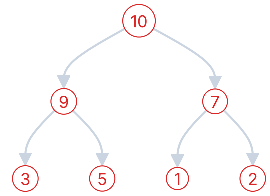
For every node i:
parent(i) <= child(i)
Root = smallest element

Max Heap

For every node i:
parent(i) >= child(i)
Root = largest element



Always true for all nodes



Always true for all nodes

5. COMMON INTERVIEW CHECKLIST

- ✅ Understand the goal clearly.
- ✅ Ask: What should I keep? What should I discard?
- ✅ Choose the right Heap (Min / Max).
- ✅ Decide the size (K or all elements).
- ✅ Maintain Heap invariant after every insertion/removal.
- ✅ Think about edge cases (empty, k = 0, k > n, duplicates).
- ✅ Analyze Time and Space Complexity.

6. COMMON MISTAKES

- ❌ Using Min Heap when Max Heap is required (or vice versa).
- ❌ Forgetting that PriorityQueue in Java is Min Heap by default.
- ❌ Not maintaining heap size while solving Top-K problems.
- ❌ Forgetting to add next element in Merge K after poll().
- ❌ Not handling empty heap before peek()/poll().
- ❌ Using subtraction in comparator (a - b) → overflow risk.
- ❌ Confusing Heap with Sorted Array / Tree.

💡 7. AHA MOMENTS

- ⭐ Heap gives you the best element in O(1).
- ⭐ It is about PRIORITY, not sorting.
- ⭐ Think: "What should I discard?" for Top-K problems.
- ⭐ Merge K = One candidate per source.
- ⭐ Median = Balance two halves.

🎯 PATTERN RECOGNITION TRIGGERS

- ➡️ **Top K / Kth Largest / Kth Smallest**
- ➡️ **Find Median / Running Median**
- ➡️ **Most Frequent / Least Frequent**
- ➡️ **Task Scheduler / CPU Scheduling**
- ➡️ **Merge K Sorted / K Sorted Lists**
- ➡️ **Closest K / Smallest Range / Meeting Rooms**

⭐ **Remember: Every Heap problem belongs to one of the 4 patterns on Page 2.**

HEAP - FAANG HIDDEN TRIGGERS

How Interviewers Hide Heap Problems (Recognition Guide)

PAGE 4

They won't say "Use Heap"
You must recognize it !

1 WHEN TO THINK HEAP ? (UNSEEN QUESTION CLUES)

Question Says...	Think Heap Because...
Top K / K largest / K smallest	We only care about best K elements.
K closest / K farthest	Keep best K based on distance / value.
Kth largest / Kth smallest	Heap gives Kth in optimal time.
Most frequent / Top frequent	Use heap on frequency map.
Continuously receive data ("data stream")	Need to maintain running result.
Running median / Median at any time	Two Heaps maintain middle efficiently.
Merge K sorted lists / arrays / streams	Pick smallest current from K sources.
Always need smallest / largest repeatedly	Heap gives min/max in O(log n) each time.
Scheduling / Highest priority task	Always process highest (or lowest) priority first.
Resource allocation	Allocate best resource (min waste / min cost).
Find N smallest among millions	Min Heap of size N.

2 FAANG FAVORITE HEAP PROBLEM THEMES

System Design Like Problems — **Heap (Max)**
Design Twitter Feed, Task Scheduler, Rate Limiter, Job Scheduler

Streaming / Real Time Data — **Two Heaps**
Running Median, Data Stream as Disjoint Intervals

Top K / Ranking — **Min Heap (size K)**
Top K Frequent, Top K Elements, K Closest Points

Merge / K Sorted Sources — **Min Heap**
Merge K Sorted Lists, Smallest Range in K Lists

Greedy + Priority — **Max Heap**
IPO, Find K Pairs with Smallest Sums, Maximize Capital

Intervals / Meetings — **Min Heap**
Meeting Rooms (III), Employee Free Time, Conference Room Scheduler

Graphs / Advanced — **Min Heap**
Dijkstra's Algorithm, Prim's MST, Network Delay Time

Pro Tip

Translate the problem into:
"Which element should come out first?"
→ **That is your heap priority.**

4 INTERVIEWER TRICKS TO WATCH OUT

Live leaderboard / Top scores	Maintain top K scores dynamically.
Sliding window maximum/minimum	(Usually Deque) but if K is large and needs frequent removal by value → Heap (advanced).

3 HEAP VS OTHER DS (CHOOSE RIGHT TOOL)

Problem Type / Goal	Best Tool	Why?
Get min / max repeatedly	**Heap**	O(log n) per operation
Top K elements	**Heap**	O(n log k) better than sorting
Kth largest / smallest	**Heap / Quickselect**	Heap: O(n log k), Quickselect: O(n) avg
Merge K sorted things	**Heap**	One candidate per source
FIFO order	Queue	Heap not for order
LIFO order	Stack	Heap not for order
Need sorted output	Sort	Full order required
Range queries	Segment Tree / BIT	Heap not for range aggregation
Dynamic min with delete by value	TreeMap / Balanced BST	Heap can't delete arbitrary element efficiently

✗ They never say "Use Heap" explicitly.

✗ They mix multiple concepts (e.g., Heap + HashMap).

✗ They change the goal (e.g., maximize instead of minimize).

✗ They give huge constraints to hint suboptimal solutions will TLE.

✗ They ask for real time / online processing → think Heap.

✗ They change K during the process (dynamic K).

✗ They ask for both min and max → think Two Heaps.

6 AHA MOMENTS (DON'T FORGET)

✓ Heap = Priority, NOT Sorting.

✓ Heap guarantees the BEST element only.

✓ Ask yourself: "What should I discard?"

✓ Heap size K keeps only what I care about.

✓ Comparator decides priority (not the data structure).

✓ **Every Heap problem fits into one of the 4 patterns on Page 2.**

5 RECOGNITION SHORTCUT (3 QUESTIONS)

1. What am I optimizing? (min/max/priority)
2. Do I need only top/bottom K or keep discarding irrelevant elements?
3. Is the data coming continuously or from multiple sorted sources?

→ If **YES** to any → **Think HEAP ✓**

⭐ **GOLDEN RULE**

The best way to solve Heap problems is to **RECOGNIZE** them fast, choose the **RIGHT HEAP**, and **MAINTAIN the INVARIANT**.

Master Recognition, Not Just Implementation !

HEAP - PATTERNS

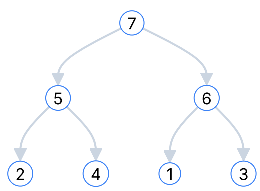
Top Interview Patterns with Heaps
(FAANG Cheat Sheet)

PAGE 2

Max Heap → largest element on top   **Min Heap** → smallest element on top   **PQ** → PriorityQueue

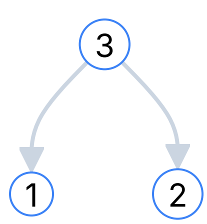
PATTERN 1 REPEATED MAX / MIN RETRIEVAL (e.g., Last Stone Weight)

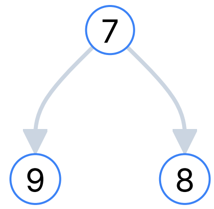
When to Use?	Heap to Use	Idea / AHA	Template / Approach (Java)	Examples

• We repeatedly take the largest (or smallest) element.
• Do some operation with it.
• Put the result back.
• Repeat until 1 (or none) left.
	Max Heap → largest element on top
	
		💡
		Always care about the current largest (or smallest) element.
		
		Heap gives it in O(1).
			`PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());`
			
			`for (int x : nums) maxHeap.offer(x);`
			
			`while (maxHeap.size() > 1) {`
			`    int a = maxHeap.poll();  // largest`
			`    int b = maxHeap.poll();  // 2nd largest`
			`    if (a != b) maxHeap.offer(a - b);`
			`}`
			
			`return maxHeap.isEmpty() ? 0 : maxHeap.peek();`
				📌 **LC 1046** Last Stone Weight
				
				Complexity
				• Time: O(n log n)
				• Space: O(n)

PATTERN 2 TOP-K ELEMENTS (e.g., Kth Largest, Top K Frequent)

When to Use?	Heap to Use	Idea / AHA	Template / Approach (Java)	Examples

• Need the K largest / smallest / most frequent elements.
• K is much smaller than n.
• We don't want to sort everything.
	Top K Largest → Min Heap (size K)
	(Keep K=3 largest seen so far)
	
	Top K Smallest → Max Heap (size K)
		💡
		Keep only K best candidates in heap.
		
		If better element comes, replace root (worst among K).
			Top K Largest (Min Heap of size K)
			`PriorityQueue<Integer> pq = new PriorityQueue<>();`
			`for (int x : nums) {`
			`    pq.offer(x);`
			`    if (pq.size() > k) pq.poll(); // remove smallest`
			`}`
			`// pq contains k largest elements`
			
			Top K Smallest (Max Heap of size K)
			`PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder());`
			`for (int x : nums) {`
			`    pq.offer(x);`
			`    if (pq.size() > k) pq.poll(); // remove largest`
				• **LC 215** Kth Largest Element in an Array
				• **LC 347** Top K Frequent Elements
				• **LC 973** K Closest Points to Origin
				• **LC 692** Top K Frequent Words
				
				Complexity
				• Time: O(n log k)



(Keep K=3 smallest seen so far)

```
}
// pq contains k smallest elements
```

- Space: O(k)

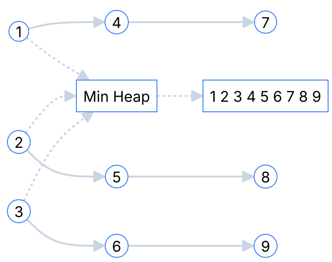
PATTERN 3 MERGE K SORTED LISTS / ARRAYS (e.g., Merge K Sorted Lists)

When to Use?

- We have K sorted lists / arrays.
- Need to merge into one sorted output.
- Always pick the smallest current candidate.

Heap to Use

Min Heap



Idea / AHA

Put the first element of each list into heap.

Pop smallest, add its next element (if any).

Repeat.

Template / Approach (Java)

```
class Node {
    int val, list, index; // value, which list, index in list
}
PriorityQueue<Node> pq = new PriorityQueue<>(
    (a, b) -> a.val - b.val
);
// 1. Add first element of each list
for (int i = 0; i < lists.length; i++) {
    pq.offer(new Node(lists[i].get(0), i, 0));
}
List<Integer> res = new ArrayList<>();
while (!pq.isEmpty()) {
    Node cur = pq.poll();
    res.add(cur.val);
    int nextIdx = cur.index + 1;
    if (nextIdx < lists[cur.list].size()) {
        pq.offer(new Node(lists[cur.list].get(nextIdx), cur.list, nextIdx));
    }
}
// res is merged sorted list
```

Examples

- **LC 23** Merge k Sorted Lists
- **LC 378** Kth Smallest Element in a Sorted Matrix
- **LC 1246** Palindrome Removal

Complexity

- Time: O(N log k) (N = total elements)
- Space: O(k)

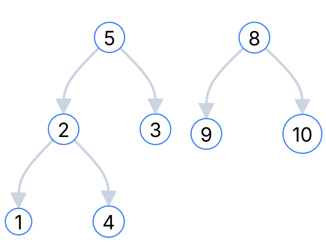
PATTERN 4 TWO HEAPS (MEDIAN FROM DATA STREAM) (e.g., Find Median from Data Stream)

When to Use?

- Need median at any point in a stream.
- Insertions happen one by one.
- Need O(log n) insert and O(1) median.

Heap to Use

Max Heap (Smaller Half) **Min Heap** (Larger Half)



Idea / AHA

Maintain two heaps:
1) maxHeap (left half) → top = max of left half
2) minHeap (right half) → top = min of right half

Template / Approach (Java)

```
PriorityQueue<Integer> maxHeap =
    new PriorityQueue<>(Collections.reverseOrder());
PriorityQueue<Integer> minHeap = new PriorityQueue<>();

void addNum(int num) {
    if (maxHeap.isEmpty() || num <= maxHeap.peek())
        maxHeap.offer(num);
    else
        minHeap.offer(num);

    // Balance
    if (maxHeap.size() -
```

Examples

- **LC 295** Find Median from Data Stream

Complexity

- Time: O(log n) per operation

		Balance size difference ≤ 1.
		
		Median is either top of maxHeap or avg of both tops.
			`minHeap.size() > 1)`
			
			`minHeap.offer(maxHeap.poll());`
			`    else if (minHeap.size() - maxHeap.size() > 1)`
			
			`maxHeap.offer(minHeap.poll());`
			`}`
			
			`double findMedian() {`
			`    if (maxHeap.size() > minHeap.size())`
			`        return maxHeap.peek();`
			`    else`
			`        return (maxHeap.peek() + minHeap.peek()) / 2.0;`
			`}`
				• Space: O(n)

🎯 **RECOGNITION TRIGGERS (If you see these in a question, think HEAP!)**

Top K / Kth Largest / Smallest	Most Frequent / Top Frequent	Merge K Sorted	Running Median / Median at any time	Continuously pick Max / Min

GOLDEN RULE:

Heaps are all about **PRIORITY, not sorting.** Choose the right heap and maintain the invariant!